

中国科学院大学

《计算机组成原理(研讨课)》实验报告

姓名 黄奕皓 学号 2024K8009929012 专业 计算机科学与技术
实验项目编号 4 实验名称 五级流水线 RISC-V 处理器设计与实现

- 注 1: 撰写此 Word 格式实验报告后以 PDF 格式保存 SERVE CloudIDE 的 /home/serve-ide/cod-lab/reports 目录下(注意: reports 全部小写)。文件命名规则: prjN.pdf, 其中 prj 和后缀名 pdf 为小写, N 为 1 至 4 的阿拉伯数字。例如: prj1.pdf。PDF 文件大小应控制在 5MB 以内。此外, 实验项目 5 包含多个选做内容, 每个选做实验应提交各自的实验报告文件, 文件命名规则: prj5-projectname.pdf, 其中“-”为英文标点符号的短横线。文件命名举例: prj5-dma.pdf。具体要求详见实验项目 5 讲义。
- 注 2: 使用 git add 及 git commit 命令将实验报告 PDF 文件添加到本地仓库 master 分支, 并通过 git push 推送到实验课 SERVE GitLab 远程仓库 master 分支(具体命令详见实验报告)。
- 注 3: 实验报告模板下列条目仅供参考, 可包含但不限定如下内容。实验报告中无需重复描述讲义中的实验流程。

一、 逻辑电路结构与设计说明

1. 寄存器堆模块

寄存器堆是处理器的通用寄存器存储载体, 遵循 RISC-V 架构规范实现 32 个 32 位宽度的通用寄存器组, 其中 x0 寄存器通过硬件组合逻辑强制恒为 0, 符合指令集架构约定。模块采用同步写、组合逻辑读的工作模式, 包含两个独立的读端口与一个写端口, 可在单周期内同时输出两个源寄存器的值, 为译码阶段提供寄存器数据读取能力, 同时在写回阶段完成目标寄存器的状态更新。

2. 算术逻辑单元(ALU)

算术逻辑单元是执行阶段的核心运算模块, 支持按位与、按位或、按位异或、按位或非、加法、减法、有符号比较、无符号比较共 8 种运算操作。模块复用同一套 32 位加法器通路, 通过补码减法原理(减数按位取反加一)实现加法、减法与大小比较运算的统一, 同时输出溢出标志、进位/借位标志、零标志三类状态信号, 既服务于常规算术逻辑指令, 也为分支指令的条件判定提供硬件依据。

3. 移位器模块

移位器为独立的专用运算通路, 支持逻辑左移、逻辑右移、算术右移三种移位模式, 可处理 0 31 位的移位操作。其中算术右移通过符号位掩码手动实现高位符号扩展, 避免调用系统移位函数带来的实现差异, 保证运算结果的确定性。该模块与 ALU 并行工作, 专门处理 R 型与 I 型移位指令, 避免移位操作占用通用 ALU 运算通路, 提升执行阶段的硬件并行度。

4. 五级流水线 CPU 核心设计

本实验实现了支持 RV32I 基础指令集的五级流水线 RISC-V 处理器, 整体划分为取指(IF)、译码(ID)、执行(EX)、访存(MEM)、写回(WB)五个流水级, 各级通过流水线寄存器实现状态隔离与传递, 支持分支跳转、非对齐访存、总线异步握手等完整功能。处理器整体结构框图如 Fig. 1 所示。

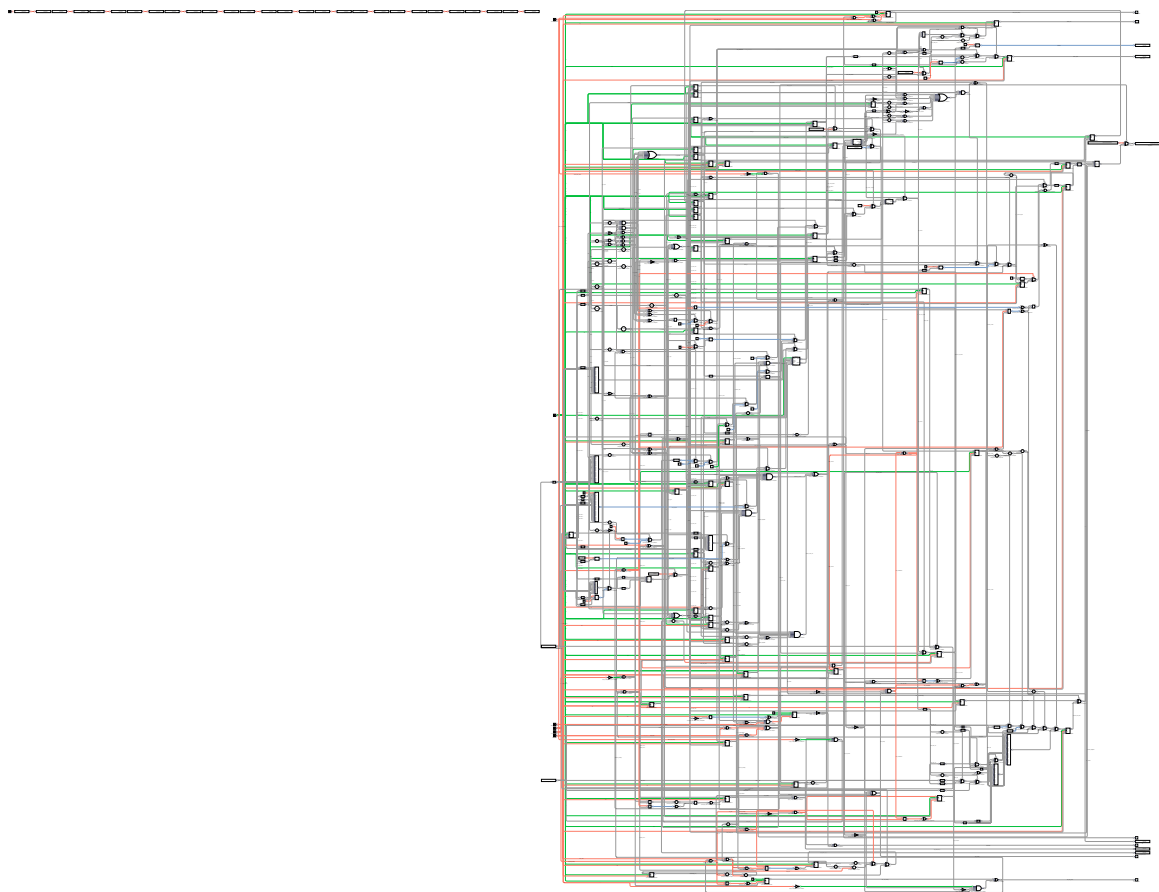


图 1: 五级流水线 CPU 整体电路图

(a) 取指级 (IF)

取指级负责程序计数器的更新与指令的总线读取,是流水线的入口阶段,核心包含 PC 更新通路、总线握手逻辑、指令有效性过滤三大子模块。

PC 更新通路采用优先级多路选择结构,按复位、分支跳转、重定向、顺序自增的优先级依次选择下一个 PC 值,核心代码逻辑如下:

```
always @(posedge clk) begin
    if (rst)
        pc_reg <= 32'h0;
    else if (branch_flush && !(Inst_Req_Valid && !Inst_Req_Ready))
        pc_reg <= ex_branch_target; // 无在途请求时直接跳转
    else if (redirect_pending && Inst_Req_Ready)
        pc_reg <= redirect_target; // 总线空闲时执行重定向
    else if (!stall_if_id && (inst_buf_valid || (inst_handshake && req_accepted)))
        pc_reg <= pc_reg + 32'd4; // 顺序取指地址自增
end
```

该结构保证分支跳转具有最高优先级,仅当总线无未完成请求时才直接更新 PC;若总线忙则锁存目标地址,待总线空闲后执行重定向,避免总线请求混乱。

总线握手逻辑采用请求保持寄存器与在途请求计数器,实现异步总线上请求与应答的精准计数;指令有效性过滤逻辑则通过丢弃计数器识别分支冲刷后的无效指令,阻止错误指令流入后续流水级。

(b) 译码级 (ID)

译码级完成指令译码、立即数扩展、寄存器堆读取与全链路控制信号生成,为纯组合逻辑电路。硬件根据指令的 opcode、funct3、funct7 字段识别指令类型,区分 R、I、S、B、U、J 六种指令格式,生成寄存器写使能、ALU 操作码、移位操作码、访存控制等全部下游控制信号,同时完成六种格式立即数的符号扩展,输出源寄存器数据与目标寄存器地址。

该级同时集成数据冒险检测逻辑,并行比对当前指令源寄存器与后三级目的寄存器的写后读依赖,核心代码如下:

```
// 分别检测EX、MEM、WB级与ID级的RAW数据冒险
wire hazard_id_ex = ifid_valid && idex_valid && idex_RegWrite && (idex_rd != 5'b0) &&
    ((rs1 == idex_rd) || (rs2 == idex_rd));
wire hazard_ex_mem = ifid_valid && exmem_valid && exmem_RegWrite && (exmem_rd != 5'b0) &&
    ((rs1 == exmem_rd) || (rs2 == exmem_rd));
wire hazard_mem_wb = ifid_valid && memwb_valid && memwb_RegWrite && (memwb_rd != 5'b0) &&
    ((rs1 == memwb_rd) || (rs2 == memwb_rd));
// 任意一级存在依赖即判定为数据冒险
assign data_hazard = hazard_id_ex || hazard_ex_mem || hazard_mem_wb;
```

检测逻辑排除了 x0 寄存器的无效依赖,只要存在任意一级的写后读依赖即输出冒险标志,用于触发流水线停顿。

(c) 执行级 (EX)

执行级是流水线的运算核心,完成算术逻辑运算、移位运算、分支目标地址计算与分支条件判定。该级通过多路选择器选择 ALU 的输入源,支持寄存器值、立即数、PC 值、常数 4 等多种输入,调用 ALU 与移位器完成对应运算。

针对分支与跳转指令,该级同时计算目标地址并判定分支条件,生成分支冲刷信号。分支条件覆盖等于、不等于、有符号小于、有符号大于等于、无符号小于、无符号大于等于六类,完全兼容 RV32I 分支

指令集规范。跳转返回地址(PC+4)也在该级生成,用于JAL、JALR指令的写回。

(d) 访存级(MEM)

访存级负责数据存储器的总线读写交互,处理非对齐访存的数据拼接与提取,是处理器与内存系统交互的接口。该级维护访存请求状态寄存器,实现与数据总线的异步握手,避免重复发送访存请求。

写操作时,硬件根据访存宽度与地址低两位生成字节使能信号,将写入数据移位到对应字节位置,适配字节、半字、字三种写入位宽;读操作时,从32位总线数据中提取目标字节或半字,再根据指令类型完成符号扩展或零扩展。

(e) 写回级(WB)

写回级是流水线的收尾阶段,负责将运算结果或访存读取的数据写回寄存器堆。该级通过二选一多路选择器选择写回数据源,支持ALU运算结果、访存读取数据两类输入,跳转指令的返回地址已在执行级并入ALU结果通路。该级生成寄存器堆的写地址、写数据与写使能信号,完成指令的最终状态更新,同时写使能信号反馈到译码级的冒险检测逻辑,用于解除对应的数据冒险。

5. 关键优化设计

(a) 指令缓存机制与总线死锁消除

核心代码片段如下:

```
reg [31:0] inst_buf; // 单条指令缓存寄存器
reg      inst_buf_valid; // 缓存数据有效标志
// 总线应答信号生成: 无效指令直接应答, 有效指令按缓存与流水线状态应答
assign Inst_Ready = stale_inst ? 1'b1 : (!inst_buf_valid || !stall_if_id);
// 有效指令握手成功标志
wire inst_handshake = real_inst_valid && Inst_Ready;

always @(posedge clk) begin
    if (rst || branch_flush) begin
        inst_buf_valid <= 1'b0;
        inst_buf <= 32'h0;
    end else if (inst_handshake && stall_if_id) begin
        // 流水线停顿且有效指令到达, 暂存至缓存并释放总线
        inst_buf_valid <= 1'b1;
        inst_buf <= Instruction;
    end else if (!stall_if_id && inst_buf_valid) begin
        // 流水线恢复运行, 消耗缓存中的指令
        inst_buf_valid <= 1'b0;
    end
end
end
```

设计意图: 解决流水线因数据冒险或访存等待发生停顿时, 指令总线返回的有效指令无法被接收, 导致总线被持续占用、系统死锁的问题。

电路行为: 当流水线停顿(stall_if_id=1)且有效指令握手成功时, 指令被写入单条目缓存, 同时总线应答信号保持有效, 提前释放总线资源; 当流水线恢复运行时, 优先从缓存中读取指令送入IF/ID寄存器, 缓存有效标志同步清零。该机制仅消耗1组32位寄存器与少量组合逻辑, 即可彻底消除流水线停顿引发的总线死锁, 提升总线交互鲁棒性。

(b) 在途指令追踪与分支冲刷完善

核心代码片段如下:

```

reg [2:0] outstanding_req; // 总线在途请求计数器
reg [2:0] discard_cnt; // 待丢弃应答计数器
// 请求/应答握手成功标志
wire req_fire = Inst_Req_Valid && Inst_Req_Ready;
wire resp_fire = Inst_Valid && Inst_Ready;

always @(posedge clk) begin
    if (rst) begin
        outstanding_req <= 3'd0;
        discard_cnt <= 3'd0;
    end else begin
        // 在途请求数：请求加1，应答减1
        outstanding_req <= outstanding_req + {2'b0, req_fire} - {2'b0, resp_fire};
        if (branch_flush) begin
            // 分支冲刷瞬间快照当前在途请求数，全部标记为待丢弃
            discard_cnt <= outstanding_req + {2'b0, req_fire} - {2'b0, resp_fire};
        end else if (resp_fire && discard_cnt > 0) begin
            // 应答到达时消耗丢弃计数
            discard_cnt <= discard_cnt - 3'd1;
        end
    end
end
// 无效指令判定：丢弃计数非零则当前指令为过期指令
wire stale_inst = (discard_cnt > 0) || (branch_flush && Inst_Valid);

```

设计意图：针对异步总线“请求发出后不可撤销”的特性，处理分支跳转时已发送到总线的在途请求，避免错误地址的指令流入流水线，造成程序执行路径错误。

电路行为：在途请求计数器实时统计总线上已发出但未应答的请求数量；分支冲刷触发时，将当前在途请求数快照存入丢弃计数器。后续每返回一次指令应答，若丢弃计数大于0则直接标记该指令为无效，不送入流水线寄存器，同时递减丢弃计数；直到计数归零后，才开始接收跳转目标地址的正确指令。该机制完整覆盖了分支跳转时的所有在途错误指令场景，保证程序流的正确性。

(c) 数据冒险的精准停顿处理

核心代码片段如下：

```

// 停顿信号：仅IF/ID级同时受数据冒险和访存等待控制
assign stall_if_id = data_hazard || mem_wait_stall;
assign stall_id_ex = mem_wait_stall;
assign stall_ex_mem = mem_wait_stall;
assign stall_mem_wb = mem_wait_stall;

// ID/EX流水线寄存器：数据冒险时插入空泡
always @(posedge clk) begin
    if (rst || flush_id_ex) begin
        idex_valid <= 1'b0;
        // 其余控制信号同步清零
    end else if (data_hazard) begin
        // 数据冒险时插入空指令，所有控制信号清零
        idex_valid <= 1'b0;
    end
end

```

```

        idex_RegWrite <= 1'b0;
        idex_MemRead <= 1'b0;
        // 其余控制信号同步清零
    end else if (!stall_id_ex) begin
        // 正常流水时传递译码级信号
        idex_valid <= id_valid;
        idex_RegWrite <= id_RegWrite;
        // 其余控制信号同步传递
    end
end
end

```

设计意图: 正确实现写后读数据冒险的处理机制, 避免全级停顿导致的流水线死锁, 保证前序指令可正常完成写回以解除冒险。

电路行为: 检测到数据冒险时, 仅停顿 IF/ID 流水线寄存器, 保持当前指令状态不变; 同时向 ID/EX 级插入空泡, 清零所有有效控制信号。EX、MEM、WB 级不受数据冒险停顿影响, 继续正常流水推进, 保证前序指令可顺利完成写回。冒险解除后, IF/ID 级恢复流水, 被阻塞的指令进入执行阶段。

(d) 共享总线的仲裁机制

核心代码片段如下:

```

// 访存等待停顿信号: 读操作等待应答、写操作等待请求握手
assign mem_wait_stall = (exmem_MemRead && exmem_valid && !Read_data_Valid) ||
    (exmem_MemWrite && exmem_valid && !Mem_Req_Ready && !mem_req_sent);

// 取指请求有效条件: 缓存空闲、无待应答请求、流水线无停顿
assign Inst_Req_Valid = inst_req_holding
    || (!rst
        && !inst_buf_valid
        && !req_accepted
        && !stall_if_id);

```

设计意图: 解决取指与数据访存共享系统总线时的访问冲突, 避免两类请求同时竞争总线导致握手失败与系统停滞。

电路行为: 数据访存具有更高的总线访问优先级。当 MEM 级存在有效访存请求且未完成时, mem_wait_stall 信号有效, 进而触发 stall_if_id 停顿, 自动屏蔽取指级的总线请求输出; 待访存请求完成握手、访存等待解除后, 取指请求恢复发送。该方案无需额外总线状态机, 仅通过流水线停顿联动实现总线仲裁, 硬件开销极低且时序性能优异。

(e) 非对齐访存的全支持

核心代码片段如下:

```

// 写通路: 数据移位与字节使能生成
wire [31:0] write_data_comb =
    is_SB ? ({24'b0, idex_read_data2[7:0]} << mem_shift) :
    is_SH ? ({16'b0, idex_read_data2[15:0]} << mem_shift) :
    is_SW ? idex_read_data2 : 32'd0;
wire [3:0] write_strb_comb =
    is_SB ? (4'b0001 << mem_addr_low) :
    is_SH ? (4'b0011 << mem_addr_low) :
    is_SW ? 4'b1111 : 4'b0000;

```

```

// 读通路：字节提取与符号/零扩展
wire [31:0] mem_load_data_comb =
    (exmem_funct3 == 3'b000) ? {{24{load_byte[7]}}, load_byte} : // LB符号扩展
    (exmem_funct3 == 3'b001) ? {{16{load_half[15]}}, load_half} : // LH符号扩展
    (exmem_funct3 == 3'b010) ? Read_data : // LW
    (exmem_funct3 == 3'b100) ? {24'b0, load_byte} : // LBU零扩展
    (exmem_funct3 == 3'b101) ? {16'b0, load_half} : Read_data; // LHU零扩展

```

设计意图：兼容 RV32I 指令集的字节、半字非对齐访存指令，支持符号扩展与零扩展两类读取模式。电路行为：写操作时，根据地址低两位将写入数据左移到对应字节位置，同时生成对应位宽的字节使能信号，实现单字节、半字的选择性写入；读操作时，根据地址低两位从 32 位总线数据中提取目标字节或半字，再根据指令类型完成符号扩展（LB/LH）或零扩展（LBU/LHU），完全符合 RISC-V 指令集的访存语义。

6. 性能评估

处理器内置两组基础硬件性能计数器，分别为时钟周期计数器（cycle_cnt）与指令退休计数器（inst_retire_cnt），对应输出 cpu_perf_cnt_0 与 cpu_perf_cnt_1，可用于流水线 CPI（每指令周期数）与 IPC（每周期指令数）的量化计算。

本设计通过 Dhrystone 与 CoreMark 两款工业标准基准测试程序完成功能验证与性能评估，两项测试均通过全部功能校验，程序执行结果完全符合预期。

表 1: Dhrystone 基准测试结果

测试项	数值
执行轮次	5 轮（每轮 100 次迭代）
各轮执行时间（微秒）	191529、191532、191529、191528、191531
平均执行时间（微秒）	191529.8
执行时间标准差（微秒）	1.64
Dhrystones per second	522
最大执行时间偏差（微秒）	4
执行时间抖动幅度	0.002%
功能校验结果	全部通过

表 2: CoreMark 基准测试结果

测试项	数值
执行轮次	5 轮（每轮 5 次迭代，CoreMark Size 666）
首轮后 4 轮平均执行时间（微秒）	81258925.5
seedcrc 校验值	0xe9f5（全部一致）
crcfinal 校验值	0xf24c（全部一致）
后 4 轮执行时间偏差范围（微秒）	161
执行时间抖动幅度	0.0002%
功能校验结果	全部通过

从测试结果可以得到以下结论：

- (a) 功能正确性: 两项测试的 CRC 校验与最终退出状态均全部通过, 验证了处理器对 RV32I 指令集的正确实现, 覆盖算术逻辑、分支跳转、访存、移位等全部指令类型。
- (b) 时序稳定性: 如 Table. 1 与 Table. 2 所示, Dhrystone 测试的执行时间抖动幅度仅为 0.002%, Core-Mark 测试的抖动幅度低至 0.0002%, 说明流水线运行高度稳定, 总线调度、冒险处理与分支冲刷逻辑均未引入随机时序波动, 处理器状态机运行可靠。
- (c) 性能分析: 当前设计的性能瓶颈主要来源于三方面: 一是写后读数据冒险引入的停顿周期, 二是分支跳转带来的两级流水线冲刷惩罚, 三是数据访存的异步等待延迟。由于当前设计仅采用停顿策略处理数据冒险, 未实现数据转发 (Forwarding) 机制, 整数运算指令间的数据依赖会引入额外停顿, 整体 CPI 高于理想五级流水线的理论值 1, 符合当前设计阶段的预期性能表现。

二、实验过程中遇到的问题及解决方法

1. 握手信号组合逻辑毛刺问题

问题现象: 初始设计中总线请求信号采用纯组合逻辑生成, 在握手临界时刻会因门电路传输延迟产生逻辑毛刺, 导致总线端误判握手状态, 处理器挂死无法继续取指。

原因分析: 异步总线的握手信号为电平敏感, 组合逻辑输出的毛刺会被总线端采样为多次握手事件, 造成请求-应答状态机错位, 在途请求计数与实际状态不一致。

对应代码修改: 新增 inst_req_holding 时序寄存器, 将请求输出信号由组合逻辑改为寄存器输出, 核心修改代码如下:

```
reg inst_req_holding;
always @(posedge clk) begin
    if (rst)
        inst_req_holding <= 1'b0;
    else if (Inst_Req_Valid && Inst_Req_Ready) // 握手成功清除保持状态
        inst_req_holding <= 1'b0;
    else if (Inst_Req_Valid && !Inst_Req_Ready) // 未握手则锁存保持状态
        inst_req_holding <= 1'b1;
end
// 请求输出由寄存器保持信号与新请求逻辑共同生成
assign Inst_Req_Valid = inst_req_holding
                        || (!rst
                           && !inst_buf_valid
                           && !req_accepted
                           && !stall_if_id);
```

修改后总线请求信号全部由寄存器输出, 消除了组合逻辑毛刺, 握手稳定性显著提升, 未再出现因毛刺导致的总线状态机错位问题。

2. 数据冒险停顿逻辑错误

问题现象: 最初实现数据冒险处理时, 将所有流水线级都设置为停顿状态, 导致出现冒险后流水线完全停滞, 前序指令无法完成写回, 数据冒险永久无法解除。

原因分析: 五级流水线处理写后读冒险的正确机制是“IF/ID 停顿 + ID/EX 插入空泡”, 后续流水级需保持前进以完成前序指令的写回。若所有级都停顿, 前序指令也会被卡住无法推进, 冒险永远无法解除, 形成死锁。

对应代码修改:调整停顿信号的生成逻辑,仅将 IF/ID 级接入数据冒险停顿,ID/EX 级改为冒险时插入空泡,核心修改如下:

```
// 修改前: 所有级都受数据冒险停顿控制
// assign stall_id_ex = data_hazard || mem_wait_stall;

// 修改后: 仅IF/ID受数据冒险控制, 其余级仅受访存等待控制
assign stall_if_id = data_hazard || mem_wait_stall;
assign stall_id_ex = mem_wait_stall;
assign stall_ex_mem = mem_wait_stall;
assign stall_mem_wb = mem_wait_stall;
```

同时在 ID/EX 寄存器的更新逻辑中增加数据冒险分支,冒险触发时直接清零所有控制信号插入空泡。修改后数据冒险可正常解除,流水线运行逻辑符合预期。

3. 取指与访存总线冲突

问题现象: 由于取指和数据访存共用同一条系统总线,当访存指令处于 MEM 级时,取指级同时发起总线请求,两者控制信号相互冲突,导致总线握手失败,系统停滞。

原因分析: 初始设计未考虑总线共享的仲裁问题,取指请求与访存请求可同时输出到总线,导致总线控制信号冲突,无法完成正常握手。先后尝试加入总线状态机、设置交替访问机制两种方案,前者状态切换延迟大,后者严重降低流水线效率,均不理想。

对应代码修改:通过访存等待信号联动取指停顿,实现访存优先级高于取指的仲裁逻辑。将 mem_wait_stall 接入 stall_if_id,访存期间自动暂停取指请求输出,待访存完成后恢复。该方案从根源上避免了两个请求同时出现在总线上的情况,逻辑简洁且额外开销极小。

4. 分支冲刷的在途指令遗漏问题

问题现象: 分支指令在 EX 级判定跳转并冲刷流水线时,已经发送到总线上的取指请求无法撤回,这些错误地址的指令后续会返回并流入流水线,导致程序执行路径错误。

原因分析: 总线具有“请求-应答”的异步特性,请求一旦发出就必须等待应答返回,不能中途取消。如果只冲刷流水线寄存器,已经发出的在途请求会在后续周期返回错误指令,造成分支跳转后程序流异常。

对应代码修改: 新增 outstanding_req 在途请求计数器与 discard_cnt 丢弃计数器,同时增加无效指令标记逻辑。分支冲刷发生时快照在途请求数存入丢弃计数器,后续返回的指令先消耗丢弃计数,标记为无效后直接丢弃,直到计数归零后才接收有效指令。该机制完整覆盖了分支延迟的在途指令场景,保证跳转后程序执行的正确性。

三、 思考题解答

1. 思考题 1: 五级流水线中数据冒险主要分为哪几类? 本设计采用了怎样的处理策略,该策略有何优缺点?

五级流水线中的数据冒险根据读写依赖关系可分为三类:写后读冒险(Read After Write, RAW)、读后写冒险(Write After Read, WAR)、写后写冒险(Write After Write, WAW)。其中 RAW 冒险是真正的数据依赖,即后序指令需要读取前序指令尚未写回的寄存器值,必须通过硬件处理;WAR 和 WAW 属于寄存器名字依赖,在五级按序发射、按序完成的流水线中不会出现,无需额外处理。

本设计采用“停顿+插入空泡”的策略处理 RAW 冒险:检测到数据依赖时,停顿 IF/ID 级的指令,在 ID/EX 级插入空操作,等待前序指令完成写回后再继续执行。该策略的优点是逻辑实现简单,功能正确性易保证,硬件开销小,易于验证与调试;缺点是会引入流水线停顿,每条数据冒险都会损失至少一个时钟周期,整体

性能较低。更优化的方案可以加入数据转发(Forwarding)逻辑,将 EX、MEM 级的结果直接转发到执行级输入端,大部分 RAW 冒险可以避免停顿,仅加载指令的一级延迟槽无法消除,可显著提升流水线性能。

2. 思考题 2: 当分支指令在 EX 级判定跳转时,流水线中已经有若干指令被取出,本设计是如何处理这些错误指令的? 请结合电路逻辑说明。

本设计通过“流水线寄存器冲刷 + 总线在途指令丢弃”两级机制完整处理分支错误指令,覆盖所有错误指令场景。

第一级是流水线寄存器冲刷: 分支跳转有效时,将 IF/ID 和 ID/EX 两级流水线寄存器全部清零,相当于插入两个空泡,已经进入流水线的两条错误指令被作废,所有有效控制信号全部置为无效,不会继续向后传播产生实际运算与访存操作。

第二级是总线在途指令丢弃: 对于已经发送到总线但尚未返回的取指请求,通过丢弃计数器记录需要丢弃的指令数量。分支冲刷发生时,快照当前总线上的在途请求数作为丢弃初始值;后续从总线返回的指令先消耗丢弃计数,被标记为无效指令直接丢弃,不进入流水线寄存器。直到丢弃计数归零后,才接收跳转目标地址的正确指令。

两级机制结合,彻底覆盖了“已进入流水线”和“已发出未返回”两类错误指令,保证分支跳转后程序流的正确性,是异步总线架构下分支处理的完整解决方案。

四、 实验所耗时间

在课后,你花费了大约 30 小时完成此次实验。